

# CRM Phase 2.5A + 2.5B

## Post-Implementation Audit Prompt

Enterprise CRM - Authentication, Tenant Isolation, Authorization, Opportunity Frontend and Vertical Slice  
Verification

*Purpose: prove whether the first real end-to-end CRM vertical slice is complete, secure, tenant-safe, architecture-consistent and ready to become the reference pattern for later modules.*

Audit / Verification Document

# How to Use This Prompt

Give this document to Claude Code after Phase 2.5A and Phase 2.5B are implemented. The task is not to add new product features. The task is to inspect the real implementation, trace the complete vertical slice, identify blockers, fix only in-scope defects, re-run verification, and produce evidence-based closure criteria.

## Context

Already completed before this phase:

- Phase 0.5 - Master Data foundation
- Phase 1 - CRM lifecycle / pipeline foundation
- Phase 1.5 - Enterprise Authorization / Access architecture
- Phase 2 - CRM Opportunity commands and APIs

## Phase 2.5 Scope

### Phase 2.5A - Authentication / Application Access Shell

- ☐ Login
- ☐ Logout
- ☐ Register
- ☐ Forgot Password
- ☐ Reset Password
- ☐ Change Password
- ☐ Session-expired handling
- ☐ Unauthorized / Forbidden screens
- ☐ Current-user / profile bootstrap
- ☐ Tenant selection / tenant switching where applicable
- ☐ Token / session refresh
- ☐ Protected routes
- ☐ Authentication loading state
- ☐ Correct post-login redirect

### Phase 2.5B - Opportunity Frontend

- ☐ Opportunity List
- ☐ Opportunity Detail
- ☐ Create Opportunity
- ☐ Edit Opportunity
- ☐ Stage Change
- ☐ Reassign
- ☐ Mark Won
- ☐ Mark Lost

The frontend must be connected to the real Phase 2 backend APIs and must not recreate authorization or domain rules as a competing client-side source of truth.

## Primary Objective

1. Determine whether Phase 2.5A is genuinely complete.
2. Determine whether Phase 2.5B is genuinely complete.
3. Verify frontend and backend contracts match correctly.
4. Verify authorization and tenant isolation remain intact.

5. Verify lifecycle and domain rules are respected.
6. Verify error, loading, empty and expired-session states are handled correctly.
7. Identify architectural shortcuts that would become technical debt.
8. Determine whether this vertical slice is safe to declare complete before the next foundation phase.

Do not assume that something works merely because the happy path works.

## Audit Principles and Scope Discipline

Preserve the existing architecture. Do not redesign the application unnecessarily.

### Do NOT

- ☐ Introduce speculative abstractions.
- ☐ Create new business features.
- ☐ Move business rules into the frontend.
- ☐ Bypass the existing authorization architecture.
- ☐ Bypass domain commands.
- ☐ Weaken tenant isolation.
- ☐ Introduce frontend-only security assumptions.
- ☐ Duplicate backend business logic in React.
- ☐ Refactor unrelated modules.

Classify every finding as BUG, SECURITY ISSUE, ARCHITECTURE ISSUE, UX GAP, TEST GAP or OPTIONAL IMPROVEMENT. Fix only findings inside the Phase 2.5A / 2.5B acceptance boundary. Document future-scope items instead of implementing them.

## 1. Phase 2.5A - Authentication Audit

Trace the complete authentication lifecycle from credentials to protected application rendering.

```
User credentials
-> authentication request
-> authenticated principal established
-> current tenant/context resolved
-> current-user/bootstrap loaded
-> permissions/capabilities available where required
-> correct destination restored or selected
-> protected application rendered
```

### 1.1 Login

- ☐ Invalid credentials
- ☐ Disabled/invalid user where supported
- ☐ Failed network/server request
- ☐ Loading state
- ☐ Repeated submit prevention
- ☐ Return URL handling
- ☐ Open-redirect vulnerability
- ☐ Already-authenticated user visiting login

### 1.2 Logout

- ☐ Server/session/token state invalidated correctly where applicable
- ☐ Client authentication state cleared
- ☐ Sensitive query/cache state cleared
- ☐ Tenant-specific cached data cleared

- ☐ Correct redirect after logout
- ☐ Browser back navigation does not expose sensitive stale screens

### 1.3 Session Expiration / Refresh

- ☐ Normal refresh
- ☐ Expired session
- ☐ Refresh failure
- ☐ Concurrent API calls during refresh
- ☐ Multiple simultaneous 401 responses
- ☐ Refresh-loop prevention
- ☐ Correct redirect to login
- ☐ Safe preservation of intended destination where appropriate

There must be no infinite refresh loops, request storms, repeated login redirects or stale authenticated UI after authentication is lost.

### 1.4 Protected Routes

- ☐ Unauthenticated users cannot enter protected areas
- ☐ Auth-loading state does not temporarily render protected content
- ☐ Protected routes do not flash sensitive UI before bootstrap completes
- ☐ Redirect behavior is deterministic

Frontend route guards are UX controls only. The backend remains the security boundary.

### 1.5 Current User / Bootstrap

- ☐ Principal/user identity
- ☐ Active tenant
- ☐ User profile
- ☐ Relevant application capabilities
- ☐ Other required application context

Bootstrap must not become tightly coupled to CRM-specific details that belong to the Opportunity module.

## 2. Tenant Switching Audit

Treat tenant switching as a security-critical operation.

```

Current Tenant A
-> switch request
-> authorization/context validation
-> Tenant B becomes active
-> tenant-dependent state invalidated
-> tenant-dependent data reloaded
-> application continues under Tenant B only

```

- ☐ TanStack/React Query tenant-dependent cache is invalidated or partitioned
- ☐ Current Opportunity detail is cleared
- ☐ Opportunity lists are cleared/reloaded
- ☐ Permissions/capabilities are refreshed when tenant-specific
- ☐ Selected organization/team state is cleared when tenant-dependent
- ☐ Stale mutations do not cross tenant boundaries
- ☐ Application state containing Tenant A information is cleared
- ☐ Tenant A data never briefly appears after switching to Tenant B
- ☐ Tenant-sensitive query keys cannot collide across tenants
- ☐ Manual route/API manipulation cannot access another tenant resource

Backend tenant isolation remains mandatory. Never trust a client-supplied tenant identifier as authorization proof.

### 3. Register / Account Creation Architecture Check

If public Register exists, verify that the identity architecture does not become dependent on self-service registration.

- ☐ Future invitation-based users remain possible
- ☐ Future administrator-created users remain possible
- ☐ Future enterprise SSO remains possible
- ☐ Future OIDC integration remains possible
- ☐ Future SAML / Entra ID or equivalent integration remains possible

Do not implement these future identity flows now. Only identify design decisions that would block them.

### 4. Opportunity List Audit

- ☐ Correct API endpoint and contract
- ☐ Tenant-safe data
- ☐ Pagination if required by existing API
- ☐ Initial loading state
- ☐ Empty state
- ☐ Error state
- ☐ Refresh/refetch behavior
- ☐ Stable query keys
- ☐ URL/filter state where implemented
- ☐ No accidental N+1 frontend requests
- ☐ No client-side reconstruction of authorization rules
- ☐ No disclosure through counts, cache, preload or metadata

### 5. Opportunity Detail Audit

- ☐ Valid Opportunity returns correct detail
- ☐ Nonexistent Opportunity produces correct 404 state
- ☐ Cross-tenant Opportunity is externally indistinguishable from nonexistent where architecture requires
- ☐ Record-level authorization denial preserves Phase 1.5 existence-hiding behavior
- ☐ Coarse authorization denial produces correct Forbidden/403 behavior where applicable
- ☐ Malformed or stale ID is handled intentionally
- ☐ Loading/retry states are intentional

If Phase 1.5 maps record-level denial to external 404 to prevent existence disclosure, the frontend must not reintroduce that information leak through messages, counts, timing assumptions or alternative endpoints.

### 6. Create / Edit Audit

- ☐ Frontend fields match backend contract
- ☐ Required fields are consistent
- ☐ Client validation improves UX but does not replace backend validation
- ☐ Backend validation errors map cleanly to form fields
- ☐ Unexpected errors use a safe generic state
- ☐ Repeated submit cannot create accidental duplicates
- ☐ Successful mutation updates/invalidates correct queries
- ☐ Failed mutation does not leave false persisted UI state
- ☐ Dirty-state handling is sensible if already implemented

## 7. Stage Change Audit

Stage Change is architecture-critical. It must use the Phase 2 lifecycle/domain command rather than treating stage as an arbitrary mutable field.

```
UI intent
-> dedicated Stage Change API/command
-> authorization
-> lifecycle validation
-> allowed transition validation
-> domain mutation
-> persistence
-> audit/result
```

- ☐ Illegal transitions are rejected by backend
- ☐ Frontend does not become authoritative transition graph
- ☐ Domain failures are displayed appropriately
- ☐ Stale/concurrent transition behavior is sensible
- ☐ List/detail cache is updated correctly
- ☐ UI hiding/disabling is only UX; backend remains authoritative

## 8. Reassign Audit

- ☐ Dedicated command/action is used if defined
- ☐ Cannot assign to invalid or inaccessible principals
- ☐ Backend validates reassignment authorization
- ☐ Dropdown visibility is not treated as authorization proof
- ☐ Record/team/tenant boundaries are enforced
- ☐ Current assignee refreshes after success
- ☐ Failure does not leave optimistic false state

## 9. Won / Lost Audit

Won and Lost are lifecycle actions, not ordinary field edits.

Mark Won

```
UI -> domain action -> authorization -> lifecycle rules -> required-data validation ->
state transition -> audit
```

Mark Lost

```
UI -> domain action -> authorization -> lifecycle rules -> Lost Reason / required metadata
-> state transition -> audit
```

- ☐ Already Won
- ☐ Already Lost
- ☐ Invalid source stage
- ☐ Repeated request
- ☐ Concurrent request
- ☐ Missing required Lost information
- ☐ Correct API/domain error handling

## 10. Authorization Matrix Test

| Scenario                                       | Expected external behavior                    |
|------------------------------------------------|-----------------------------------------------|
| Authenticated + authorized + record accessible | Success                                       |
| Not authenticated                              | 401 / authentication flow                     |
| Authenticated but no coarse capability         | 403 where architecture defines it             |
| Capability exists but record inaccessible      | 404 where Phase 1.5 requires existence hiding |
| Different tenant                               | Never disclose record                         |
| Invalid/nonexistent ID                         | 404                                           |
| Unauthorized field/action                      | Rejected                                      |
| UI hides action but direct API is attempted    | Backend still rejects                         |

Inspect real code and tests; do not treat the matrix as merely theoretical.

## 11. Error Contract Audit

- ☐ 400 / malformed request
- ☐ Validation error
- ☐ 401
- ☐ 403
- ☐ 404
- ☐ Conflict / concurrency if supported
- ☐ Domain rule violation
- ☐ Unexpected 5xx
- ☐ Network unavailable

Do not collapse every failure into the same generic toast if meaningful user action is possible. Do not expose stack traces, internal exception names, SQL/database information, resource existence information or authorization internals.

## 12. Frontend Authorization UX

Review action visibility for Create, Edit, Reassign, Stage Change, Won and Lost.

- ☐ Capability information is used to avoid showing obviously unavailable actions
- ☐ Every action is still protected server-side
- ☐ No role-name shortcuts such as `user.role === "Admin"` contradict the capability/action model
- ☐ Frontend authorization is treated as UX, not enforcement

## 13. Loading / Empty / Error States

### Opportunity List

- ☐ Initial loading
- ☐ Loaded with records
- ☐ Loaded empty
- ☐ Failed
- ☐ Refetching

## Opportunity Detail

- ☐ Loading
- ☐ Loaded
- ☐ Not found
- ☐ Forbidden where applicable
- ☐ Failed

## Forms / Mutations

- ☐ Idle
- ☐ Submitting
- ☐ Success
- ☐ Validation failure
- ☐ Domain failure
- ☐ Network/server failure

No expected application state should result in an ambiguous blank page.

## 14. Query / Cache Audit

- ☐ Query key consistency
- ☐ Tenant included wherever necessary
- ☐ List/detail separation
- ☐ Mutation invalidation
- ☐ Stale data behavior
- ☐ Duplicate requests
- ☐ Cache leakage
- ☐ Over-invalidation
- ☐ Under-invalidation

Explicitly verify these state transitions:

- Create Opportunity -> list reflects it
- Edit Opportunity -> list and detail reflect it
- Stage Change -> list and detail reflect it
- Reassign -> list and detail reflect it
- Won/Lost -> list and detail reflect it
- Tenant switch -> old tenant data disappears immediately and safely
- Logout -> authenticated data cache is cleared

## 15. Security Review

- ☐ Cross-tenant data leaks
- ☐ IDOR / insecure direct object access
- ☐ Trusting tenant ID sent by client
- ☐ Frontend-only authorization enforcement
- ☐ Stale authorization state
- ☐ Cache leakage
- ☐ Unsafe return URLs
- ☐ Token leakage
- ☐ Sensitive token storage
- ☐ Sensitive logging
- ☐ XSS-prone rendering

- ☐ CSRF concerns according to actual auth approach
- ☐ Exposed internal exception information

Do not invent theoretical vulnerabilities. Tie findings to actual code paths and observed behavior.

## 16. Backend / Frontend Contract Consistency

- ☐ Duplicated handwritten models that have drifted
- ☐ Nullable mismatch
- ☐ Enum mismatch
- ☐ Date/time mismatch
- ☐ ID type mismatch
- ☐ Pagination mismatch
- ☐ Error-contract mismatch
- ☐ Casing/serialization assumptions
- ☐ Fields the frontend assumes but backend does not guarantee

Do not introduce code generation unless already part of project architecture or clearly justified. If useful, document it as a future recommendation instead.

## 17. Responsive / Application-Shell Sanity Check

- ☐ Application shell works at common desktop widths
- ☐ Forms remain usable
- ☐ Detail/list layouts remain navigable
- ☐ Loading overlays cannot block indefinitely
- ☐ Dialogs can be dismissed
- ☐ Keyboard focus is reasonable
- ☐ Lifecycle/destructive actions use suitable confirmation if current UX design calls for it

Do not turn this audit into a full UI redesign.

## 18. Test Coverage

Inspect existing tests and prioritize high-value coverage around security and lifecycle boundaries.

1. Authentication lifecycle
2. Tenant isolation
3. Coarse vs record authorization
4. Opportunity lifecycle commands
5. Forbidden direct API attempts
6. Tenant switch cache isolation
7. Won/Lost
8. Reassign
9. Stage transitions
10. Validation/error mapping

Prefer meaningful integration/end-to-end coverage over large numbers of trivial unit tests. If Playwright or equivalent is already configured, use it for critical user journeys. Do not introduce a new E2E framework solely for this audit unless justified.

## 19. End-to-End User Journeys

**Journey A:** Login -> Opportunity List -> Opportunity Detail -> Logout

**Journey B:** Login -> Create Opportunity -> appears in list -> open detail  
**Journey C:** Open Opportunity -> Edit -> Save -> refreshed state correct  
**Journey D:** Opportunity -> valid Stage Change -> correct new state  
**Journey E:** Opportunity -> Reassign -> new owner reflected correctly  
**Journey F:** Opportunity -> Mark Won -> lifecycle state correct  
**Journey G:** Opportunity -> Mark Lost -> reason/required data -> lifecycle state correct  
**Journey H:** Tenant A -> view data -> switch Tenant B -> verify no Tenant A data remains visible/cached  
**Journey I:** Session expires during authenticated usage -> safe refresh/re-auth flow -> no security/data leak  
**Journey J:** Direct API access to Opportunity outside user record scope -> correct secure response

## 20. Expected Architecture State After Phase 2.5

```

Browser / React
-> Application Shell
-> Authentication Context
-> Tenant Context
-> Frontend capability-aware UX
-> Typed API boundary
-> Backend Authentication
-> Tenant Resolution
-> Coarse Authorization
-> Tenant-safe Resource Load
-> Record Authorization
-> Field / Action constraints where implemented
-> Domain Command
-> Lifecycle / Business Rules
-> Persistence
-> Audit

```

The frontend must never become an alternative business-rule or authorization layer.

## 21. Definition of Done

- ☐ Authentication lifecycle works end to end.
- ☐ Logout clears sensitive client state.
- ☐ Expired sessions are handled safely.
- ☐ Protected routes do not leak protected UI/data.
- ☐ Tenant switch cannot leak previous-tenant data.
- ☐ Opportunity CRUD/action flows use real backend APIs.
- ☐ Authorization semantics from Phase 1.5 remain intact.
- ☐ Stage Change uses domain lifecycle behavior.
- ☐ Reassign uses domain authorization.
- ☐ Won/Lost are explicit lifecycle actions.
- ☐ Validation and domain errors render intentionally.
- ☐ Loading/empty/not-found/forbidden/error states are covered.
- ☐ Direct API calls cannot bypass UI restrictions.
- ☐ Critical query caches update correctly.
- ☐ No obvious cross-tenant/IDOR issue exists.
- ☐ High-value tests cover security and lifecycle boundaries.
- ☐ No Phase 2.5 blocker remains.

## 22. Execution Mode

1. Inspect architecture and relevant files.
2. Trace Phase 2.5A authentication flows.
3. Trace tenant-context behavior.
4. Trace each Phase 2.5B Opportunity flow from UI to domain/database.
5. Inspect authorization enforcement.
6. Inspect cache/query behavior.
7. Inspect error-state handling.
8. Inspect tests.
9. Run relevant tests/build/typecheck/lint according to project conventions.
10. Reproduce important issues where practical.
11. Fix only genuine Phase 2.5 blockers and high-confidence defects.
12. Re-run verification.
13. Produce the final report.

Do not make broad unrelated refactors.

## 23. Final Report Format

### 1. Executive Result

Choose exactly one: PASS / PASS WITH NON-BLOCKING FINDINGS / FAIL - BLOCKERS REMAIN. Do not use PASS unless the acceptance criteria are actually met.

### 2. Phase 2.5A Status

For each major auth capability, report PASS / PARTIAL / FAIL with evidence.

### 3. Phase 2.5B Status

For List, Detail, Create, Edit, Stage Change, Reassign, Won and Lost, report PASS / PARTIAL / FAIL with evidence.

### 4. Security / Tenant Isolation Findings

Separate Critical, High, Medium and Low findings. Use severity only when technically justified.

### 5. Architecture Findings

Explain violations of tenant-safe loading, authorization boundaries, domain-command boundaries, frontend/backend responsibility, lifecycle handling or application-shell boundaries.

### 6. Fixes Made

For every change: problem, root cause, files/components affected, solution and verification evidence.

### 7. Remaining Non-Blocking Findings

Document future-scope work without implementing it.

### 8. Verification Evidence

Include actual commands/tests/checks and results.

### 9. Final Architecture State

Briefly explain authentication -> tenant -> authorization -> Opportunity frontend -> API -> domain.

## 10. Phase 2.5 Completion Decision

Explicitly state whether Phase 2.5A + 2.5B can be considered CLOSED. If not, list exact blockers.

**Final directive:** The goal is not to claim that the implementation is good. The goal is to prove whether the first real enterprise CRM vertical slice is correct, secure, coherent and ready to become the reference pattern for subsequent modules.